Estructura de Datos y Algoritmos

Estructuras de datos

Facultad de
Ingeniería

Universidad de La Sabana

VIGILADA MINEDUCACIÓN



Objetivos

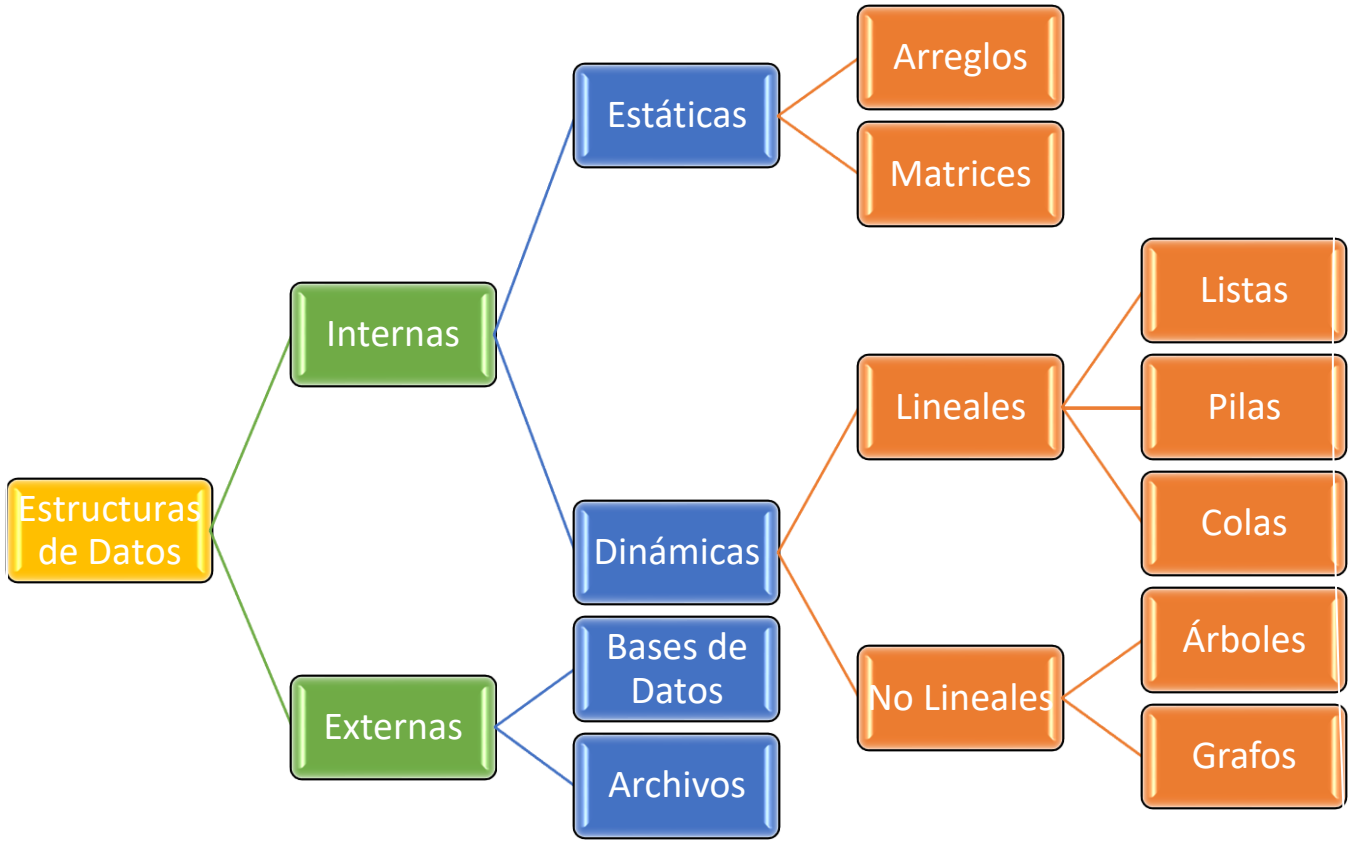
Las estructuras de datos son el corazón de la programación eficiente. Elegir la correcta no solo impacta en la velocidad de un programa, sino en su capacidad para escalar y adaptarse a nuevas necesidades.

- **Conectar teoría y práctica:** Resolver problemas usando las estructuras adecuadas para cada caso.
- **Pensar como un ingeniero:** Analizar, elegir y justificar la estructura de datos óptima según el contexto.



¿Qué son las estructuras de datos?

Es una forma particular de organizar información en un computador para que pueda ser utilizada de manera eficiente.

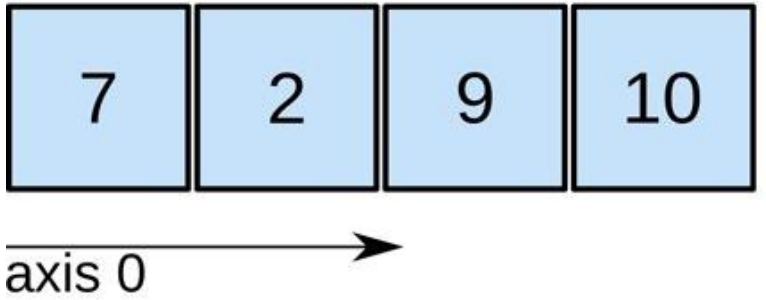



Estáticas



Arreglos

Colección de elementos del mismo tipo, almacenados de forma contigua en memoria.



Características:
- Acceso por índice en O(1).
- Tamaño fijo.
- Mejor rendimiento para acceso secuencial o aleatorio.

Operaciones comunes y complejidad:
- Acceso: O(1)
- Búsqueda: O(n)
- Inserción/eliminación: O(n)

Aplicaciones:
- Almacenamiento de registros de tamaño fijo.
- Buffers de datos.

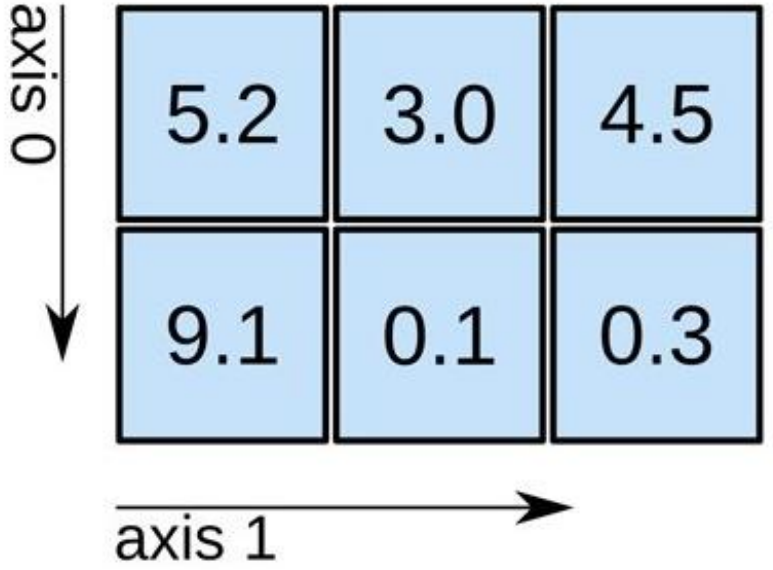
Ejemplo en Python:

```python
import array
arr = array.array('i', [1, 2, 3, 4])
print(arr[2])  # Accede al elemento en índice 2
```



Matrices

Estructura bidimensional (arreglo de arreglos) donde los datos se organizan en filas y columnas.



Características:

- Acceso mediante dos índices [fila][columna].
- Usadas en matemáticas, gráficos, imágenes, y procesamiento de datos.

Operaciones comunes y complejidad:

- Acceso: O(1)
- Recorrido completo: O(m × n)

Aplicaciones:

- Representación de tableros de juego.
- Procesamiento de imágenes.
- Operaciones algebraicas.

Ejemplo en Python:

```python
matriz = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
]
print(matriz[1][2])  # Accede a 6
```



Multidimensionales



Platzi

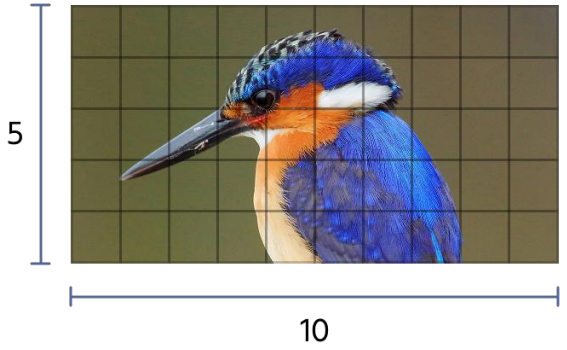


Usos comunes

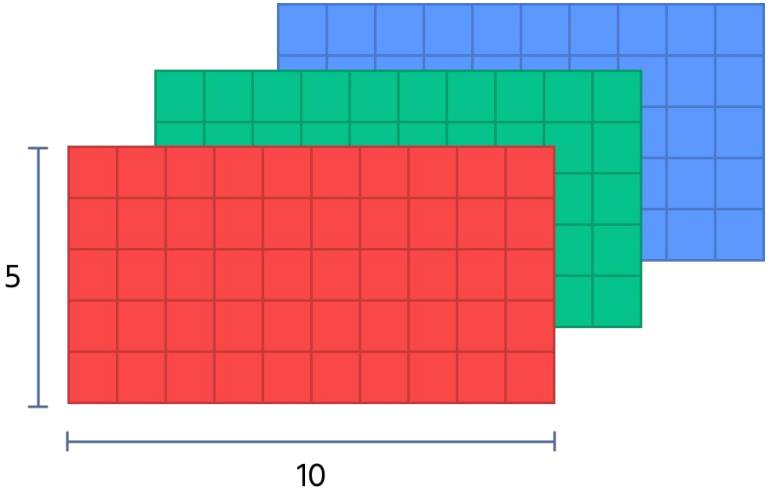
Inteligencia Artificial / Machine Learning:
Los tensores son la estructura de datos central en librerías como TensorFlow o PyTorch.

Procesamiento de imágenes:
Una imagen RGB puede verse como un tensor 3D (alto, ancho, canales).

Cálculos físicos y gráficos:
En física computacional y simulaciones, los tensores representan propiedades en múltiples direcciones.



Original Color Image



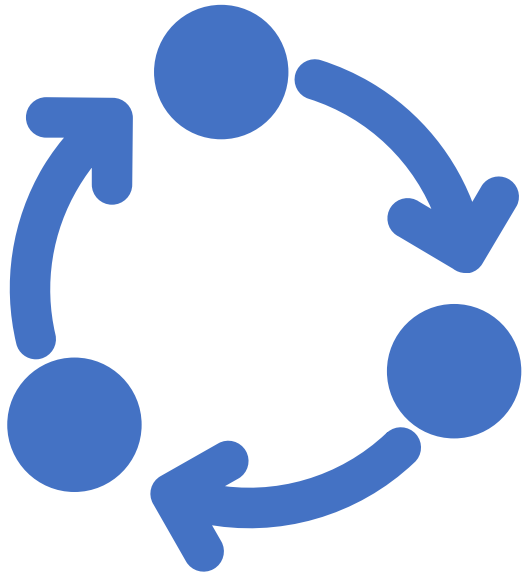
3D RGB Tensor

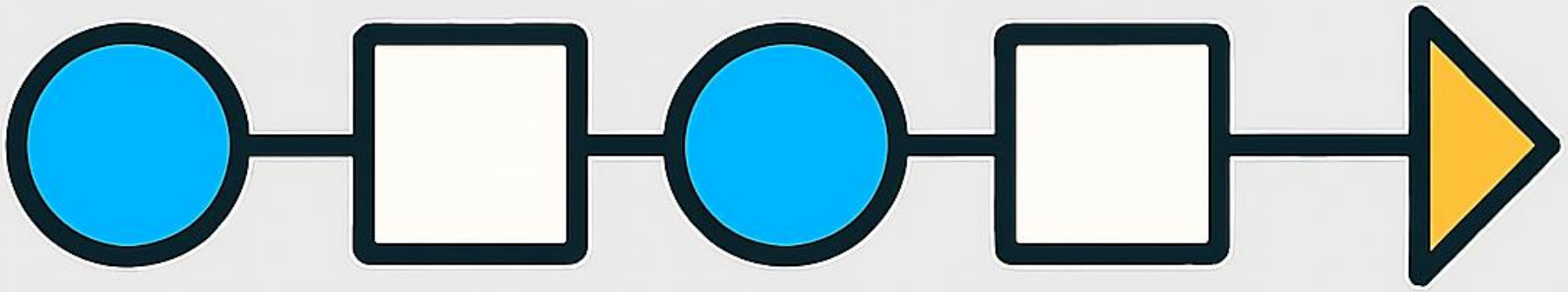
Dinámicas

Definición

Son estructuras que:

- Permiten **inserción y eliminación** de elementos en tiempo de ejecución sin un tamaño fijo predefinido.
- Mantienen un orden lógico secuencial (lineal).
- Utilizan **memoria dinámica** (asignada y liberada según necesidad).





LINEALES



Listas

Una lista es una secuencia ordenada de elementos modificable.

Operaciones Básicas:

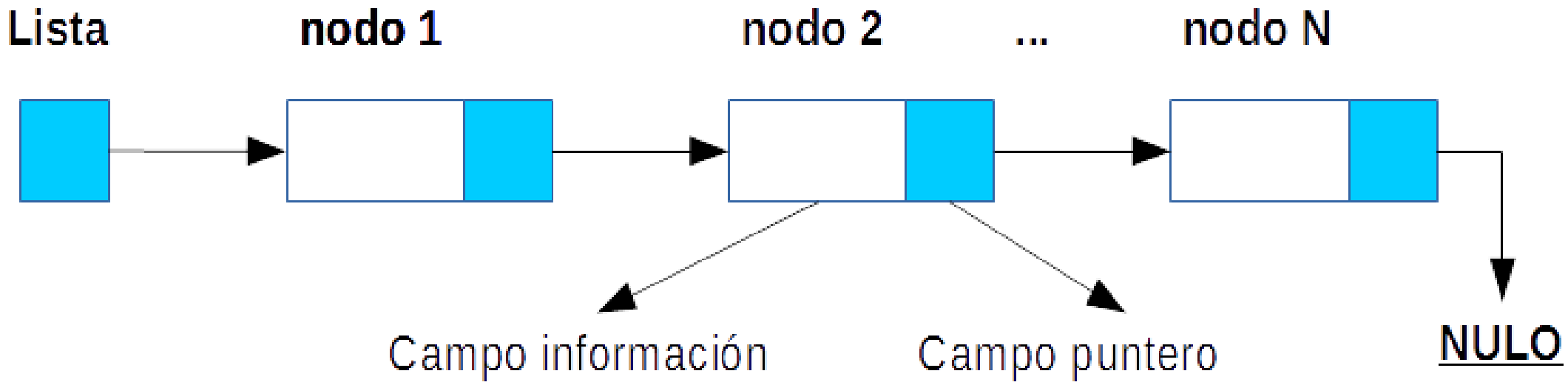
- ✓ Agregar (append): Agrega un elemento al final de la lista.
  - lista.append(elemento)
- ✓ Insertar (insert): Inserta un elemento en una posición específica.
  - lista.insert(posición, elemento)
- ✓ Acceder (indexing): Accede a un elemento por su índice.
  - elemento = lista[indice]
- ✓ Eliminar (remove): Elimina un elemento específico.
  - lista.remove(elemento)
- ✓ Longitud (len): Retorna la longitud de la lista.
  - longitud = len(lista)

Complejidad Temporal Promedio:

- ✓ Agregar al final: O(1)
- ✓ Insertar en una posición específica: O(n)
- ✓ Acceder por índice: O(1)
- ✓ Eliminar un elemento: O(n)



Lista simplemente enlazada





Pilas y Colas

Una lista es una secuencia ordenada de elementos modificable.

Operaciones Básicas:

Pilas
- ✓ Agregar (push): Agrega un elemento al final de la lista.
  - pila.append(elemento)
- ✓ Eliminar (pop): Elimina un elemento específico.
  - elemento = lista.pop(elemento)

Colas
- ✓ Agregar (enqueue): Agrega un elemento al final de la lista.
  - cola.append(elemento)
- ✓ Eliminar (enqueue): Elimina un elemento específico.
  - elemento = lista.pop(0)

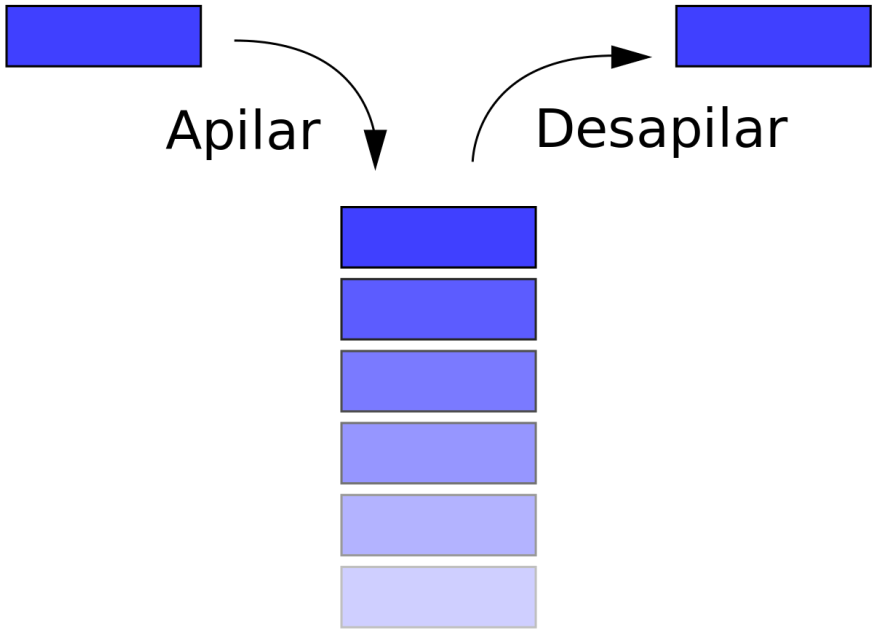
Complejidad Temporal Promedio:

- ✓ Agregar al final: O(1)
- ✓ Eliminar un elemento: O(1)



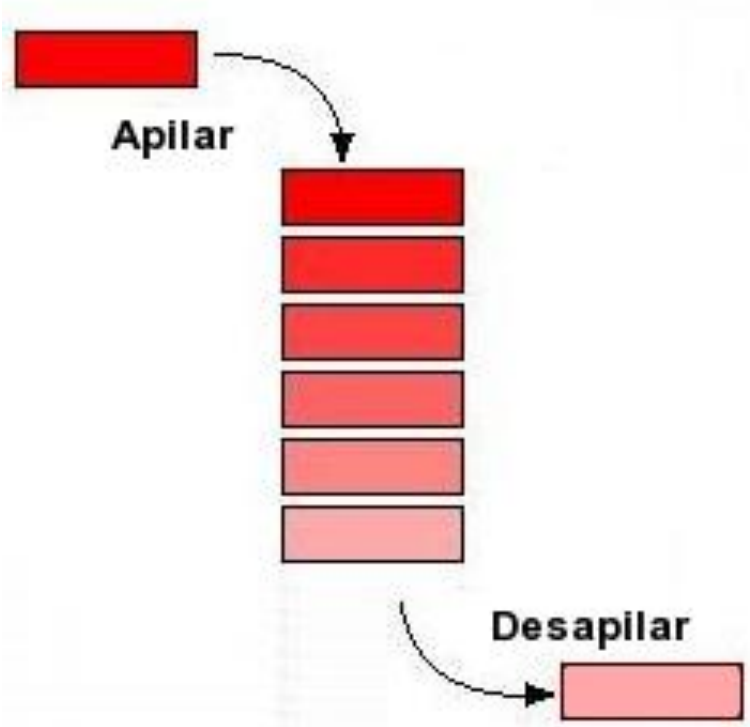
LAST IN, FIRST OUT

LIFO



FIRST IN, FIRST OUT

FIFO



```python
# Implementación de una pila usando lista
pila = []

# Agregar elementos (push)
pila.append("A")
pila.append("B")
pila.append("C")

print("Pila:", pila)

# Eliminar elementos (pop) - LIFO
ultimo = pila.pop()
print("Elemento eliminado:", ultimo)
print("Pila después de pop:", pila)
```

Salida esperada:

```
Pila: ['A', 'B', 'C']
Elemento eliminado: C
Pila después de pop: ['A', 'B']
```

📌 Último en entrar ("C") es el primero en salir.

```python
from collections import deque

# Implementación de una cola usando deque
cola = deque()

# Agregar elementos (enqueue)
cola.append("A")
cola.append("B")
cola.append("C")

print("Cola:", cola)

# Eliminar elementos (dequeue) - FIFO
primero = cola.popleft()
print("Elemento atendido:", primero)
print("Cola después de dequeue:", cola)
```

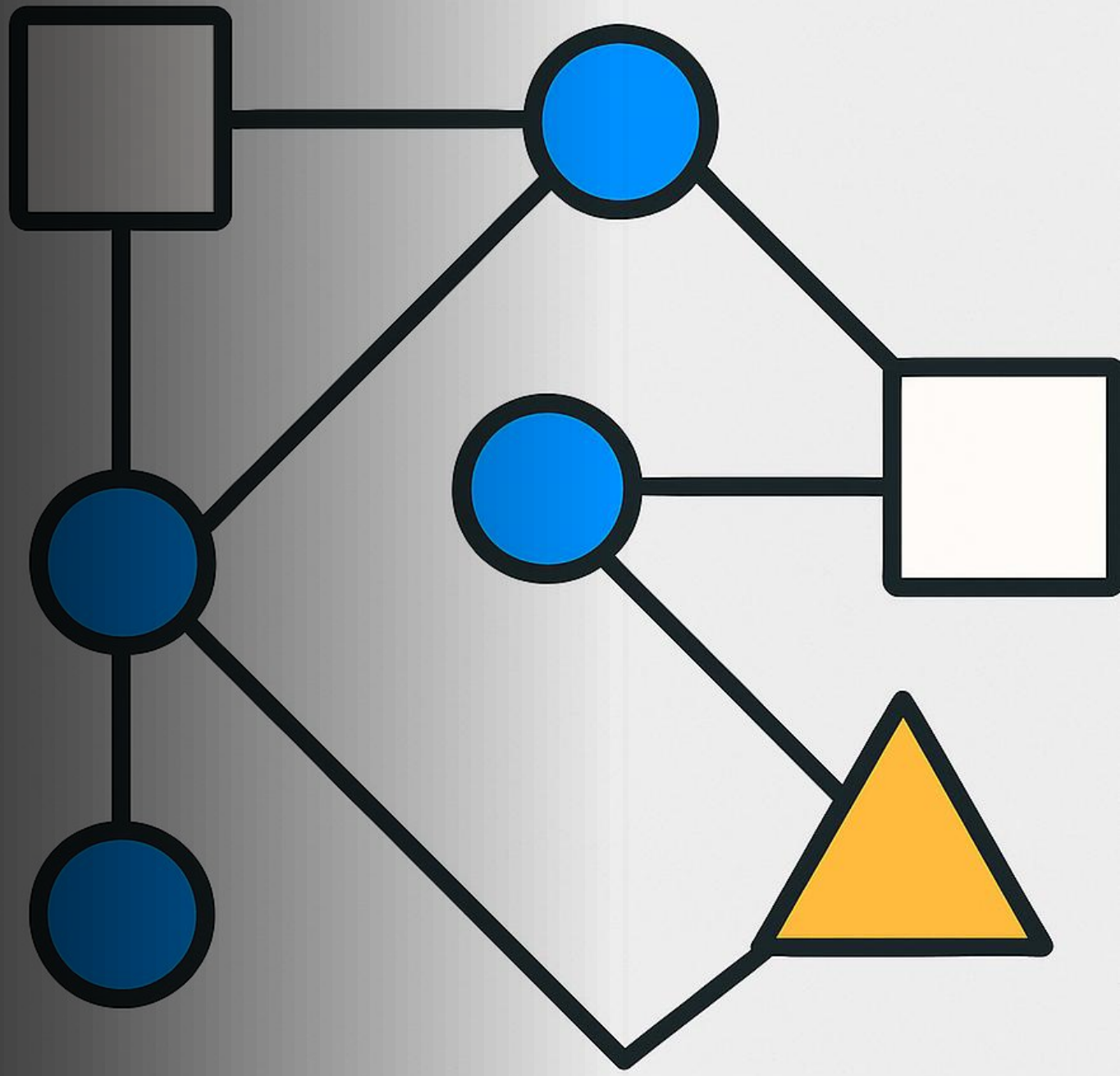
Salida esperada:

```
Cola: deque(['A', 'B', 'C'])
Elemento atendido: A
Cola después de dequeue: deque(['B', 'C'])
```

📌 Primero en entrar ("A") es el primero en salir.

Estructura	Acceso	Inserción	Eliminación	Memoria
Lista enlazada	O(n)	O(1) si se conoce nodo	O(1) si se conoce nodo	Más que un arreglo (punteros)
Pila	O(n) para acceso aleatorio	O(1)	O(1)	Baja
Cola	O(n) para acceso aleatorio	O(1)	O(1)	Baja

No lineales



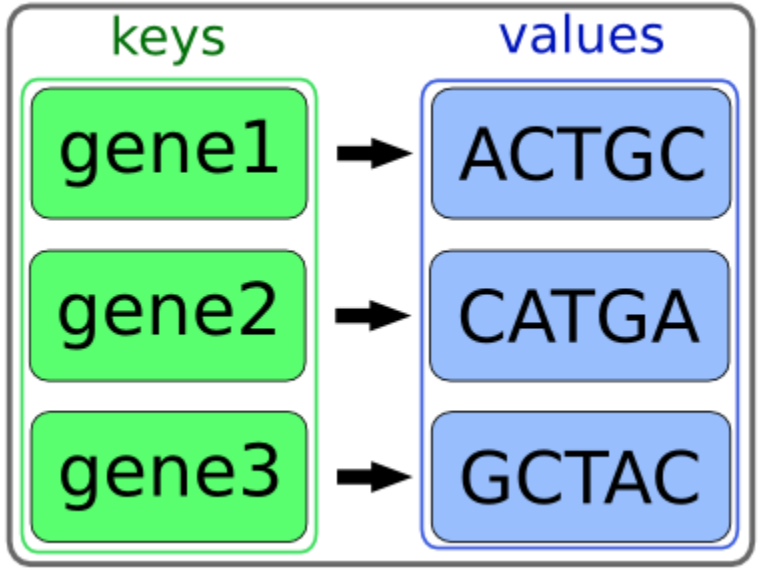


Diccionario

Un diccionario es una colección no ordenada de pares clave-valor

Operaciones Básicas:

- ✓ Agregar/Actualizar (append/update): Agrega o actualiza un elemento clave / valor. $\theta(1)$
  - diccionario[clave] = valor
- ✓ Acceder (get): Accede a un elemento por su índice.$\theta(1)$
  - valor = diccionario.get(clave)
- ✓ Eliminar (remove): Elimina un elemento específico.$\theta(1)$
  - del diccionario[clave]





Diccionario - operaciones

```python
# Crear un diccionario
estudiantes = {
    "Juan": 4.5,
    "Ana": 3.8,
    "Luis": 4.2
}

# Acceder por clave
print(estudiantes["Ana"])       # 3.8
print(estudiantes.get("Pedro")) # None si no existe
```

```python
# Acceder por clave
print(estudiantes["Ana"])       # 3.8
print(estudiantes.get("Pedro")) # None si no existe

# Agregar / actualizar
estudiantes["Pedro"] = 4.0

# Eliminar
del estudiantes["Luis"]

# Recorrer claves y valores
for nombre, nota in estudiantes.items():
    print(nombre, "→", nota)
```

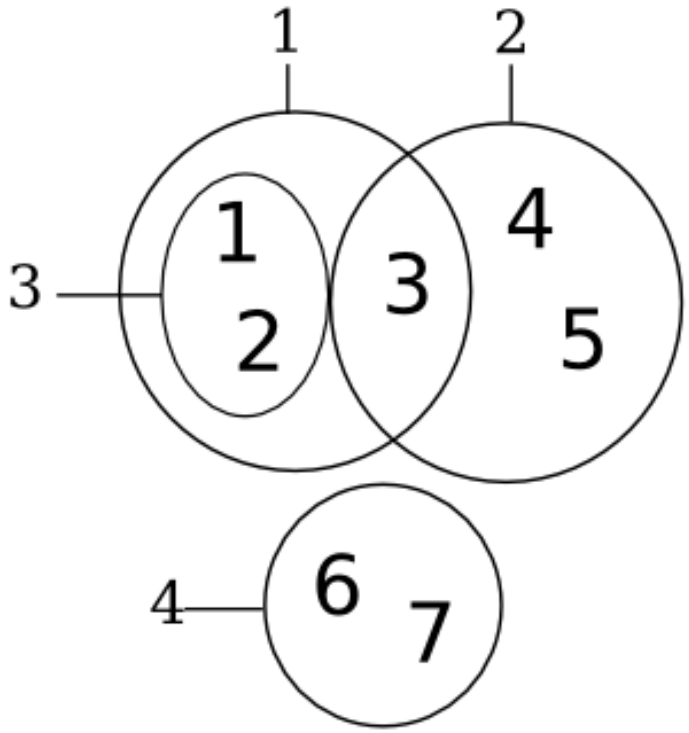


Conjuntos

Un conjunto es una colección desordenada y sin elementos duplicados.

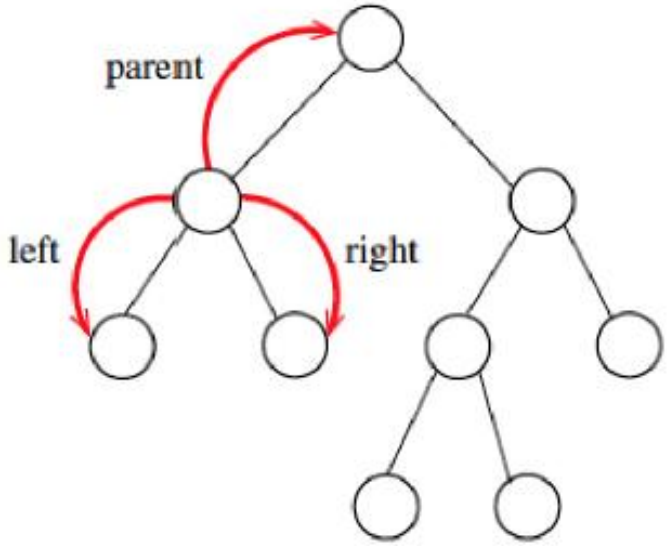
Operaciones Básicas:

- ✓ Agregar (append): Agrega un elemento al final de la lista. $O(1)$
  - conjunto.add(elemento)
- ✓ Eliminar (remove): Elimina un elemento específico. $O(1)$
  - conjunto.remove(elemento)
- ✓ Operaciones con conjuntos: Operaciones algebraicas de conjuntos $O(\min(\text{len}(\text{conjunto1}), \text{len}(\text{conjunto2})))$
  - c = a.union(b)

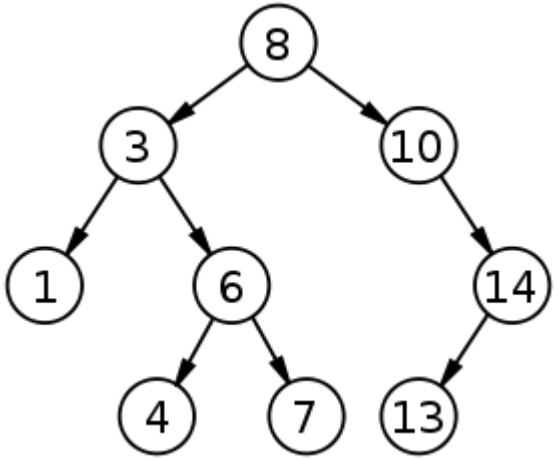




Arboles binarios



- Los árboles de búsqueda binaria proporcionan una estructura de datos que soporta de manera eficiente las operaciones de búsqueda, inserción, borrado, mínimo, máximo, sucesor y predecesor.
- Un árbol binario es un árbol enraizado donde cada nodo contiene como máximo dos hijos.
- Cada hijo puede identificarse como un hijo izquierdo o derecho.

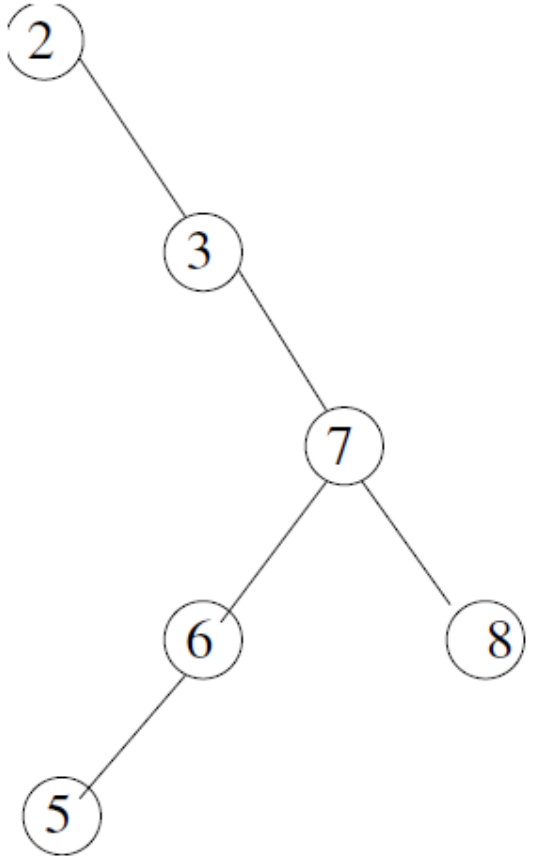




Arboles binarios

Un árbol de búsqueda binaria etiqueta cada nodo x en un árbol binario de modo que todos los nodos en el subárbol:

- Izquierdo de x tienen claves $< x$
- Derecho de x tienen claves $> x$.





Implementación del Árbol Binario

Cada nodo tiene un valor asociado que puede ser cualquier tipo de dato (entero, cadena, objeto, etc.). Este valor es lo que se almacena en el nodo del árbol.

```
class Nodo:
    def __init__(self, valor):
        self.valor = valor
        self.izquierda = None
        self.derecha = None
```



Búsqueda en un Árbol Binario

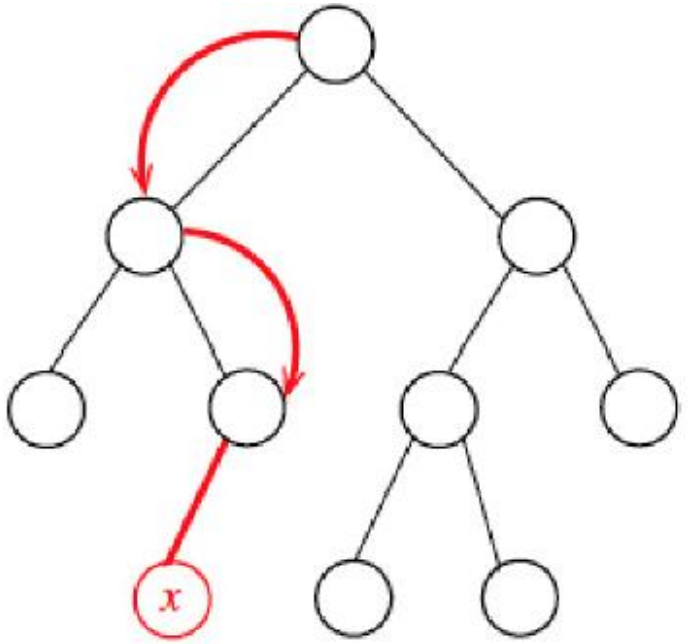
```
def buscar(self, valor):
        return self._buscar_recursivo(valor, self.raiz)

def _buscar_recursivo(self, valor, nodo_actual):
        if nodo_actual is None:
                return False
        if valor == nodo_actual.valor:
                return True
        elif valor < nodo_actual.valor:
                return self._buscar_recursivo(valor, nodo_actual.izquierda)
        else:
                return self._buscar_recursivo(valor, nodo_actual.derecha)
```



Inserción en un Árbol Binario

- Haga una búsqueda binaria para encontrar dónde debería estar, luego reemplace el puntero de terminación **None** con el nuevo elemento.
- La inserción lleva un tiempo proporcional a la altura del árbol: O(h)





Inserción en un Árbol Binario

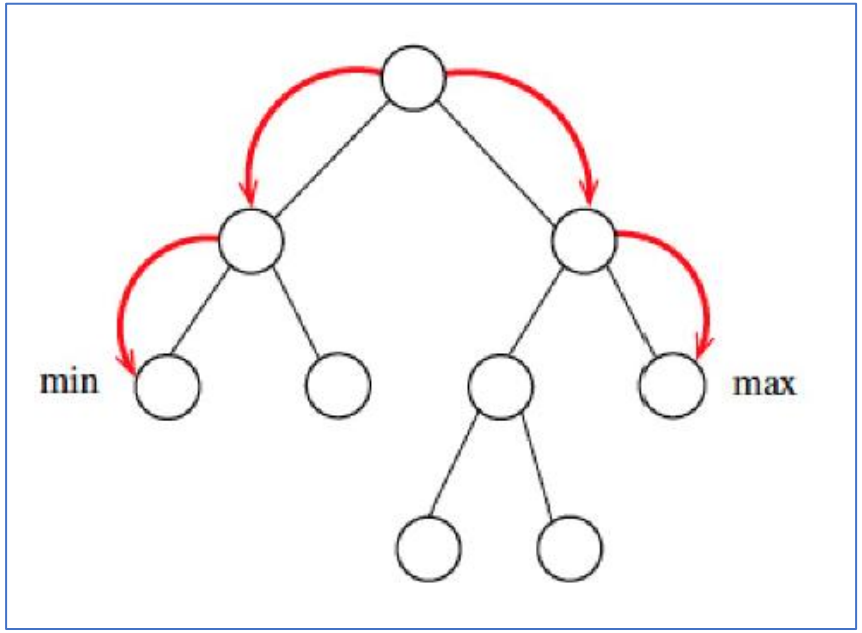
```
def insertar(self, valor):
        if self.raiz is None:
                self.raiz = Nodo(valor)
        else:
                self._insertar_recursivo(valor, self.raiz)
```

```
def _insertar_recursivo(self, valor, nodo_actual):
        if valor < nodo_actual.valor:
                if nodo_actual.izquierda is None:
                        nodo_actual.izquierda = Nodo(valor)
                else:
                        self._insertar_recursivo(valor, nodo_actual.izquierda)
        else:
                if nodo_actual.derecha is None:
                        nodo_actual.derecha = Nodo(valor)
                else:
                        self._insertar_recursivo(valor, nodo_actual.derecha)
```



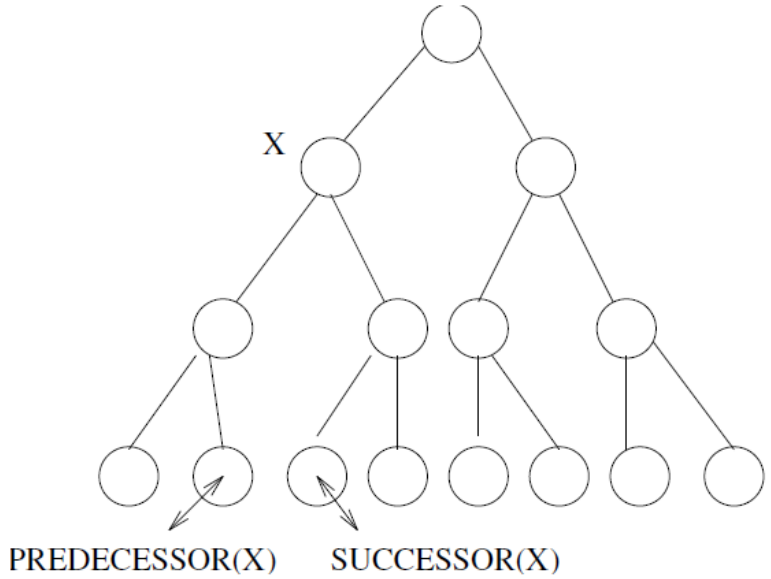
Máximo y Mínimo

- ¿Dónde se encuentran los valores máximo y mínimo de un árbol de búsqueda binaria?
- Encontrar el máximo o el mínimo lleva un tiempo proporcional a la altura del árbol, O(h).





Predecesor y sucesor

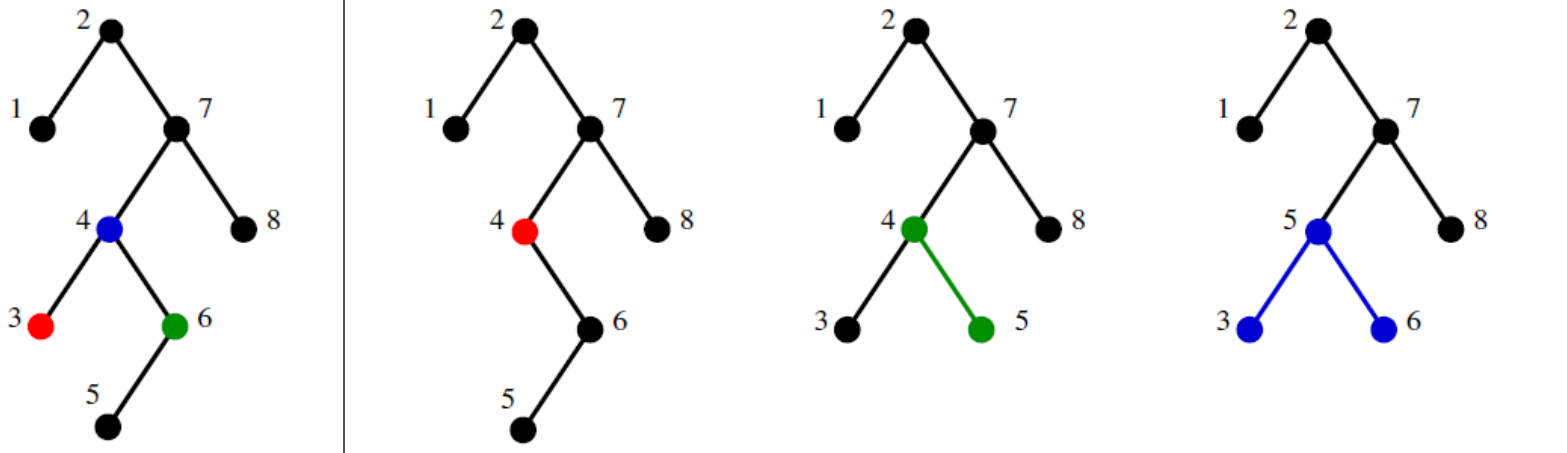


Si X tiene dos hijos, su predecesor es el valor máximo en su subárbol izquierdo y su sucesor el valor mínimo en su subárbol derecho.



Borrado en Árbol binario

La eliminación es más complicada que la inserción, porque el nodo que va a morir puede no ser una hoja y, por lo tanto, afectar a otros nodos.



initial tree

delete leaf node (3)

delete node with 1 child (6)

delete node with 2 children (4)



Tipos de ordenamiento

Preorden: (raíz, izquierdo, derecho).

Se deben realizar las siguientes operaciones recursivamente en cada nodo, comenzando con el nodo de raíz:

1. Visite la raíz
2. Atraviese el sub-árbol izquierdo
3. Atraviese el sub-árbol derecho

Inorden: (izquierdo, raíz, derecho).

Se deben realizar las siguientes operaciones recursivamente en cada nodo, comenzando con el nodo de raíz:

1. Atraviese el sub-árbol izquierdo
2. Visite la raíz
3. Atraviese el sub-árbol derecho

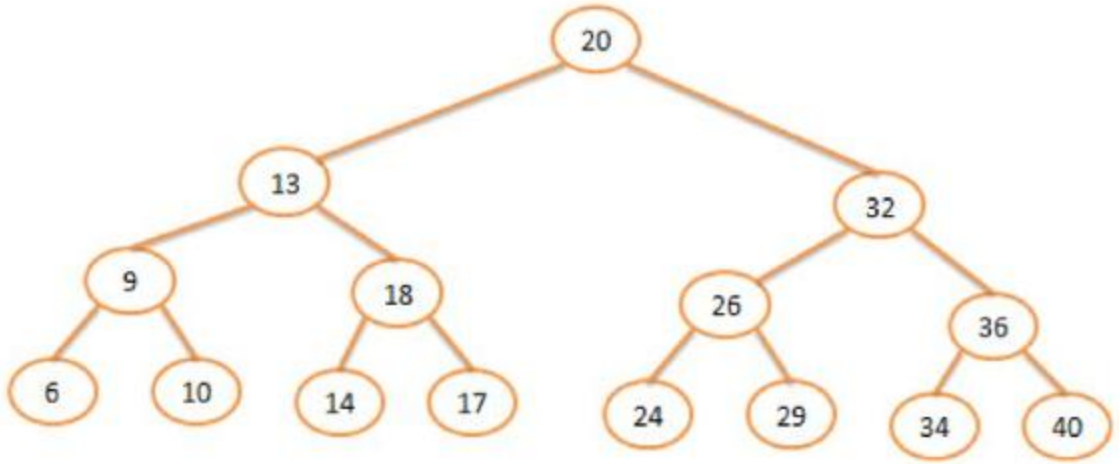
Postorden: (izquierdo, derecho, raíz).

Se deben realizar las siguientes operaciones recursivamente en cada nodo, comenzando con el nodo de raíz:

1. Atraviese el sub-árbol izquierdo
2. Atraviese el sub-árbol derecho
3. Visite la raíz



Ejercicio





Ejercicio 2

Implemente el algoritmo para poder borrar un nodo sin afectar el árbol binario

Facultad de
Ingeniería



Universidad de
La Sabana